

CS 1101 Unit 6

Programming Assignment

Assignment Response

1. Employee and Salary List Operations

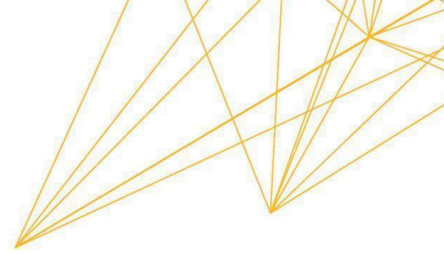
Consider that you are working as Data Analyst in an organization. The HR department needs you to carry out following operation on existing list of 10 employees name (you can assume 10 names in a list here).

1. Split the list into two sub-list namely subList1 and subList2, each containing 5 names.
2. A new employee (assume the name “Kriti Brown”) joins, and you must add that name in subList2.
3. Remove the second employee's name from subList1.
4. Merge both the lists.
5. Assume there is another list salaryList that stores salary of these employees. Give a rise of 4% to every employee and update the salaryList.
6. Sort the SalaryList and show top 3 salaries.

```
# Part 1: Employee and Salary List Operations
```

```
def employee_management():
```

```
    # 1. Initial list of 10 employees
```



```
employees = ["Alice", "Bob", "Charlie", "David", "Eve", "Frank", "Grace", "Heidi", "Ivan", "Judy"]
```

```
print("Original List:", employees)
```

```
# 2. Split the list into subList1 and subList2 (5 names each)
```

```
subList1 = employees[:5]
```

```
subList2 = employees[5:]
```

```
print("SubList 1:", subList1)
```

```
print("SubList 2:", subList2)
```

```
# 3. Add "Kriti Brown" to subList2
```

```
subList2.append("Kriti Brown")
```

```
print("SubList 2 after addition:", subList2)
```

```
# 4. Remove the second employee's name from subList1 (Index 1)
```

```
del subList1[1]
```

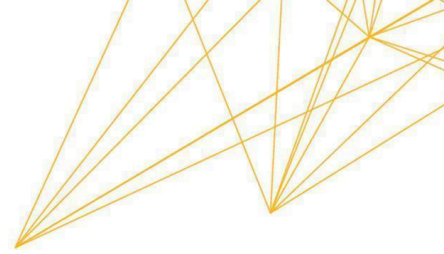
```
print("SubList 1 after removal:", subList1)
```

```
# 5. Merge both lists
```

```
mergedList = subList1 + subList2
```

```
print("Merged List:", mergedList)
```

```
# 6. Salary List: Give a 4% rise to every employee
```



```
salaryList = [50000, 52000, 48000, 60000, 55000, 58000, 47000, 51000, 53000, 59000]

# We use a list comprehension to apply the 4% raise (current * 1.04)

salaryList = [round(salary * 1.04, 2) for salary in salaryList]

print("Updated Salary List (4% rise):", salaryList)


# 7. Sort the SalaryList and show top 3 salaries

salaryList.sort(reverse=True)

print("Top 3 Salaries:", salaryList[:3])


# Main execution

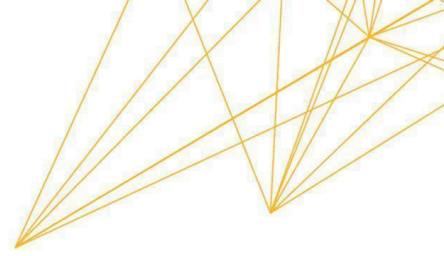
if __name__ == "__main__":

    employee_management()
```

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 6 | main ?1 | python3 employeeListOperations.py
Original List: ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy']
SubList 1: ['Alice', 'Bob', 'Charlie', 'David', 'Eve']
SubList 2: ['Frank', 'Grace', 'Heidi', 'Ivan', 'Judy']
SubList 2 after addition: ['Frank', 'Grace', 'Heidi', 'Ivan', 'Judy', 'Kriti Brown']
SubList 1 after removal: ['Alice', 'Charlie', 'David', 'Eve']
Merged List: ['Alice', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy', 'Kriti Brown']
Updated Salary List (4% rise): [52000.0, 54080.0, 49920.0, 62400.0, 57200.0, 60320.0, 48880.0, 53040.0, 55120.0, 61360.0]
Top 3 Salaries: [62400.0, 61360.0, 60320.0]
```

Technical Explanation

The program begins by initializing an array of strings. As the textbook states, ***“Like a string, a list is a sequence of values. In a string, the values are characters; in a list, they can be any type”*** (Downey, 2015, p. 89). To split this sequence, the code utilizes the slice operator (:), since



“The slice operator also works on lists” (Downey, 2015, p. 91). We slice the first five items into subList1 and the remaining into subList2.

Next, we modify the lists by appending "Kriti Brown" and deleting the second element of subList1 at index 1. This is possible because *“Unlike strings, lists are mutable”* (Downey, 2015, p. 90). The lists are then merged using the addition symbol, because *“The + operator concatenates lists”* (Downey, 2015, p. 91).

Finally, we process the salaryList. To give a 4% raise, the code uses a list comprehension, which acts as a "map" processing pattern. The text defines a map as *“A processing pattern that traverses a sequence and performs an operation on each element”* (Downey, 2015, p. 100). We multiply each element by 1.04. The .sort(reverse=True) method arranges the elements in descending order, and slicing [:3] extracts the top three salaries.

200 words

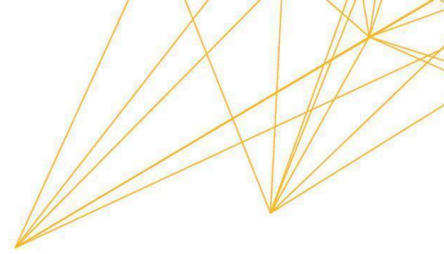
2. String to Wordlist Conversion and Reversal

Design a program such that it converts a sentence into wordlist. Reverse the wordlist then. Write the code and output for the same.

```
# Part 2: String to Wordlist Conversion and Reversal

def string_operations():

    sentence = "Learning Python is fun and productive"
```



```
# Convert sentence into wordlist

wordlist = sentence.split()

print("\nWord List:", wordlist)


# Reverse the wordlist

wordlist.reverse()

print("Reversed Word List:", wordlist)


# Main execution

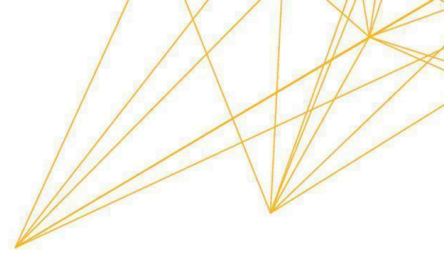
if __name__ == "__main__":

    string_operations()
```

```
• ~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 6 | main ?1 python3 stringToWordlistReversal.py
Word List: ['Learning', 'Python', 'is', 'fun', 'and', 'productive']
Reversed Word List: ['productive', 'and', 'fun', 'is', 'Python', 'Learning']
```

Technical Explanation

In this section, a variable `sentence` is initialized with a string of text. To convert this string into a list of individual words, the `.split()` method is applied. As noted in the course material, ***“If you want to break a string into words, you can use the split method”*** (Downey, 2015, p. 95). By default, this method uses spaces as the delimiter to determine where the string should be split.



Once the string is converted into a list format, the `.reverse()` method is called on `wordlist`.

Because lists are mutable, this method changes the list in place, effectively flipping the order of the elements so that the last word becomes the first, and so on.

115 words

References

Downey, A. (2015). Think Python: How to think like a computer scientist. Green Tree Press.

<https://greenteapress.com/thinkpython2/thinkpython2.pdf>